Counting with the Derivative Tower

The Budan–Fourier theorem, machine-checked in Lean 4

Carles Marín*
Independent researcher
karlesmarin@gmail.com

June 2026

Abstract

I report a complete, `sorry`-free formalization in Lean 4, over Mathlib, of the *Budan–Fourier theorem*: for a nonzero real polynomial p and an interval $(a, b]$ whose endpoints are not roots, the number of real roots of p in $(a, b]$, *counted with multiplicity*, is at most the drop $\mathrm{V}(a) - \mathrm{V}(b)$ in the sign variations of the derivative tower p, p', p'', $\ldots$, $p^{(\deg p)}$, and differs from it by an even number:

$$\#\mathrm{roots}(a, b] \;\le\; \mathrm{V}(a) - \mathrm{V}(b), \qquad \mathrm{V}(a) - \mathrm{V}(b) - \#\mathrm{roots}(a, b] \text{ is even.}$$

No root is ever located; signs along a tower of derivatives are read and subtracted. The mathematics is classical and has been formalized once before, in Isabelle/HOL by Wenda Li [9]; to the best of my knowledge—based on searches of Loogle and Mathlib in June 2026 (Section 8)—this is the first proof of the Budan–Fourier theorem in Lean. Mathlib carries Descartes' rule of signs (`Polynomial.signVariations`, the coefficient count) but neither the derivative tower nor the interval theorem; this work fills that gap and is the exact companion of a recent Lean formalization of Sturm's theorem [15], sharing its sign-variation toolkit. The contribution is the formalization and the reusable machinery the proof required: two one-step *monotonicity bricks* that fix the sign of a derivative just off a root, an abstract pair of list operations (`Rseq`/`Lseq`) that decouples the sign-bookkeeping from the polynomials, and the identification of the leading-zero run of the tower with the root multiplicity. The headline theorem `budan_fourier` depends only on the three standard axioms `propext`, `Classical.choice`, `Quot.sound`.

What you will find here. This is a short guided tour, not a reference manual, and it is built like a string of beads: each section ends by handing the reader the question the next one answers. The first asks how a difference of two integers can bound—and nearly count—the roots (Section 2); the second, why the count moves only at critical points, and there by the *multiplicity*, up to an even surplus (Section 3); the third, where the proof genuinely resists being made formal, now that the tower is no longer a coprime chain (Section 4); the fourth, what becomes free once the theorem is in the library—including why Descartes' rule overcounts by an even number (Section 5). The load-bearing lemmas are gathered in one table (Section 7); the honest limits are stated without varnish in Section 8; and the closing section asks, by way of reflection, the questions I would rather carry into the next paper than pretend to have settled. If you read only two things, read Theorem 1 and Figure 2; the rest is the argument that connects them.

*Formalized with the assistance of an AI system (Claude, Anthropic); the mathematics is classical, and every claim, scope statement, and limitation is the author's responsibility. The boundary of the contribution is set out plainly in Section 8, and the Lean kernel—not the assistant—is what certifies the proofs.

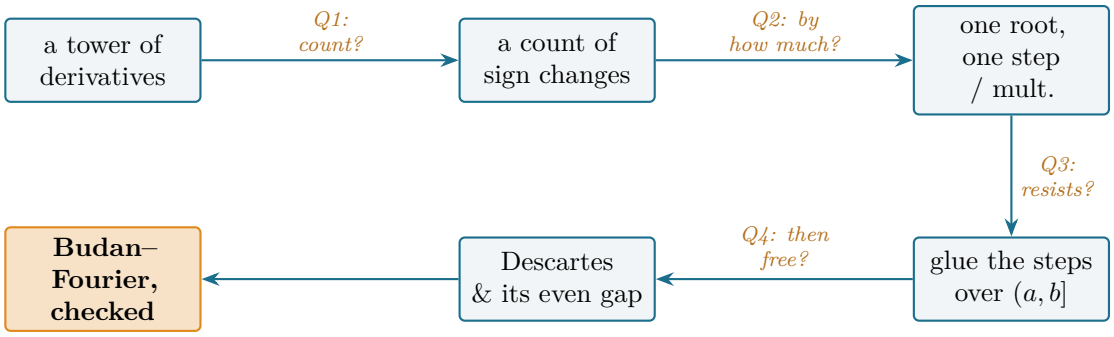

Figure 1: The reader's map. From the derivative tower to a certified interval count and what it yields downstream; each arrow is the question its section answers. Every node is a block of `sorry`-free Lean.

1 The question that begins it

How many real roots does a polynomial have between here and there—and can you say so without finding a single one of them? Descartes' rule of signs (1637) [1] reads a *bound* on the positive roots off the signs of the coefficients. In the early nineteenth century François Budan [2] and Joseph Fourier [3] sharpened it from the half-line to an arbitrary interval, and from the coefficients to the *derivative tower*: line up

$$p,\ p',\ p'',\ \ldots,\ p^{(\deg p)},$$

read the number $\mathrm{V}(x)$ of sign changes of this list at a point x (zeros discarded), and the drop $\mathrm{V}(a) - \mathrm{V}(b)$ across $(a, b]$ is an upper bound for the number of roots there. Sturm [4] later closed the gap between bound and count, but with a *different* sequence—the negated Euclidean remainders—and only for distinct roots; the Budan–Fourier statement, on the tower and with multiplicity, kept two features Sturm's did not: it counts each root as often as its order, and its bound differs from the true count by an *even* number. That even gap is not noise. It is the fingerprint of the complex roots the interval cannot see, and it is the reason Descartes' rule, the $b \to \infty$ shadow of this theorem, overcounts by an even amount and never by an odd one.

The theorem is classical and has been machine-checked once, in Isabelle/HOL, by Wenda Li [9], who also used it to drive a verified root-counting procedure and extended Sturm's theorem to multiplicity along the way. What it had not been is checked in *Lean*. Mathlib carries Descartes' rule—the function `Polynomial.signVariations`, the coefficient count—but not the derivative-tower sequence and not the interval theorem. This paper fills that gap. It is, to be exact, a first-in-Lean and not a first-in-any-system;[1] the value is in adding the natural companion of Sturm's theorem to a library that lacked it, and in the reusable pieces the proof leaves behind.

I came to it the same way I came to Sturm [15]: by asking which classical results in real-root counting a mature library is still missing. With Sturm's theorem in place and its sign-variation toolkit already written, Budan–Fourier was the next conspicuous absence—and, as it turned out, the one whose local analysis is genuinely *harder* than Sturm's, for a reason worth the trip. The rest of the paper is that trip, told as the four questions a careful reader asks in turn.

[1] Priority claim based on searches of Loogle and a `grep` of Mathlib as of June 2026; the prior Isabelle/HOL formalization is credited in Section 8.

2 Q1: how can a sign difference bound, and nearly count, the roots?

History bead. The tower, the interval count, and the parity refinement are Budan's and Fourier's ([2, 3]; the modern textbook account is Basu–Pollack–Roy [8], Ch. 2, and the historical record is disentangled by Akritas [5]). Nothing in this section is new mathematics; the Lean is in `BudanFourier.lean`, the definitions `fourierSeq` and `fourierVar`.

Let us make the two ingredients precise. The *Fourier sequence* of p is its derivative tower,

$$\texttt{fourierSeq}\,p \;=\; [\,p,\; p',\; p'',\; \ldots,\; p^{(\deg p)}\,],$$

a `List` of polynomials of length $\deg p + 1$. The count is

$$\mathrm{V}_p(x) \;=\; \texttt{fourierVar}\,p\,x,$$

the number of sign changes in the list of evaluated signs after deleting the zeros. A single `rfl` records that this is the same construction the Sturm development already analyzed, applied to a different list: `fourierVar p x = Sturm.signVarAt (fourierSeq p) x`. That one line lets the entire toolkit from the Sturm paper—local constancy on a root-free interval, the recursive sign-change count `signChanges`, the leading-pair recursion—act verbatim on the tower.

I will not re-derive that toolkit here; it is the subject of the companion paper [15]. The two are deliberately *different* papers, and it is worth saying plainly why. Sturm's theorem runs on the negated-remainder chain, counts *distinct* roots, and is an exact equality whose local analysis touches only one pair of the chain. The Budan–Fourier theorem here runs on the *derivative tower*, counts roots *with multiplicity*, is an *inequality* with an even-parity refinement, and—because the tower is not a coprime chain—its local analysis must handle a whole *vanishing block* at once. **Same toolkit underneath, genuinely different theorem on top:** this paper imports the shared machinery by citation and spends its pages only on what the tower forces that the chain did not.

Theorem 1 (Budan–Fourier, in Lean 4; `budan_fourier`)**.** *Let p be a nonzero real polynomial and $a < b$ with $p(a) \neq 0$ and $p(b) \neq 0$. Then, writing $\#\mathrm{roots}(a,b]$ for the number of real roots of p in $(a,b]$ counted with multiplicity,*

$$\#\mathrm{roots}(a,b] \;\le\; \mathrm{V}_p(a) - \mathrm{V}_p(b) \quad \textit{and} \quad \mathrm{V}_p(a) - \mathrm{V}_p(b) - \#\mathrm{roots}(a,b] \;\textit{is even.}$$

The statement is `sorry`-free; `#print axioms BudanFourier.budan_fourier` reports only `propext`, `Classical.choice`, `Quot.sound`.

That a difference of two integers should control a root count is, on first sight, a small miracle, so it is worth seeing it happen by hand—and seeing both of its faces, multiplicity and the even gap, in one example each.

A hand-checkable case: multiplicity. Take $p_A(x) = (x-1)^2(x+1) = x^3 - x^2 - x + 1$, with a simple root at -1 and a double root at 1. The tower is

$$p_A, \quad p_A' = 3x^2 - 2x - 1, \quad p_A'' = 6x - 2, \quad p_A''' = 6,$$

and reading the signs at the ends of $(-2, 2]$ gives $\mathrm{V}(-2) = 3$ and $\mathrm{V}(2) = 0$, so $\mathrm{V}(-2) - \mathrm{V}(2) = 3$, exactly the root count *with multiplicity* (1 at $x = -1$, 2 at $x = 1$). The staircase of V (Figure 2(A)) drops by one at the simple root and by *two* at the double root: **the step height is the root's**

multiplicity. Here the inequality of Theorem 1 is an equality. Table 1 reads the example off point by point; every entry is re-checked in exact arithmetic by the figure script.

A hand-checkable case: the even surplus. Take $p_B(x) = x^2 + 1$, with no real root at all. Its tower is $p_B,\ 2x,\ 2$, and $\mathrm{V}(-2) = 2$ while $\mathrm{V}(2) = 0$, so the bound is $\mathrm{V}(-2) - \mathrm{V}(2) = 2$ while the true count is 0. **The surplus is 2—even.** Looking closer (Figure 2(B)), the staircase drops at $x = 0$, which is a zero of the *interior* member $p_B' = 2x$ but not a root of p_B: the drop happens at a critical point that no root explains, and it comes in a block of even size because p_B's missing roots are a conjugate pair. This is the phenomenon Descartes' rule exhibits globally; Budan–Fourier localizes it.

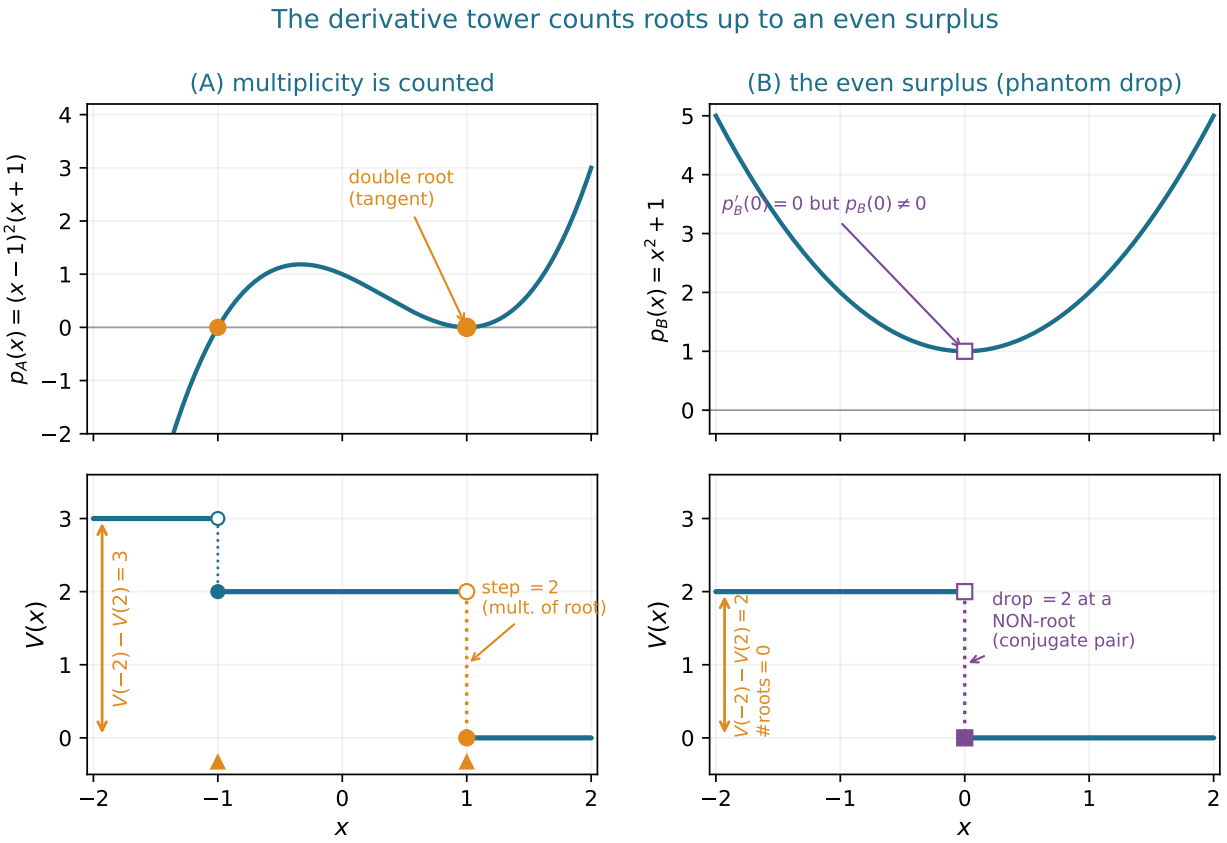

Figure 2: The derivative tower counts roots up to an even surplus. *(A)* $p_A = (x-1)^2(x+1)$: V drops by 1 at the simple root $x = -1$ and by 2 at the double root $x = 1$ (open circle = the value just to the left, filled = the value at the point, on the lower step). The descent over $(-2, 2]$ equals the root count *with multiplicity*, so the inequality is sharp. *(B)* $p_B = x^2 + 1$, no real root: V still drops by 2, at $x = 0$, where the interior member p_B' vanishes but p_B does not—the even surplus, the signature of a conjugate pair. Every plotted sign and count is asserted against exact arithmetic before drawing.

The picture behind the tables is the staircase that names the paper's method. Plot $\mathrm{V}(x)$ as x sweeps the line: it is flat almost everywhere and drops only at *critical points*—points where some member of the tower vanishes. At a root of p it drops by the multiplicity; at a critical point that is not a root of p it can still drop, by an even amount. The whole content of the next two sections is in proving that the staircase has exactly this shape. That is the bead this section hands on: *why*

x	p_A	p'_A	p''_A	p'''_A	$\mathrm{V}(x)$
	x^3-x^2-x+1	$3x^2-2x-1$	$6x-2$	6	
-2	$-$	$+$	$-$	$+$	3
$-\frac{3}{2}$	$-$	$+$	$-$	$+$	3
-1 (root)	0	$+$	$-$	$+$	2
0	$+$	$-$	$-$	$+$	2
$\frac{1}{2}$	$+$	$-$	$+$	$+$	2
1 (double root)	0	0	$+$	$+$	0
2	$+$	$+$	$+$	$+$	0

Table 1: Example A read off the tower. V falls by 1 at the simple root $x = -1$ and by 2 at the double root $x = 1$: the drop counts multiplicity. On $(-2, 2]$, $\mathrm{V}(-2) - \mathrm{V}(2) = 3$ equals the root count with multiplicity, so the Budan–Fourier inequality is sharp here. The whole table is regenerated and checked in exact arithmetic by `budan_fig1_tower.py`.

does V *move only at critical points, and there by precisely the multiplicity plus an even number?*

3 Q2: why does the count move only at critical points, and by how much?

History bead. The local analysis is the heart of every proof of Budan–Fourier. What makes it harder than Sturm's is structural: Sturm's chain is built so that consecutive members are coprime, so no two of them vanish together and the analysis at a root touches only the leading pair (p, p'). The derivative tower has no such luck—p and p' *do* vanish together at a multiple root, and a whole *block* of consecutive members $p^{(i)}, \ldots, p^{(j)}$ can vanish at one point. The Lean is in the lemmas `sign_eval_right_vanish`, `sign_eval_left_vanish` (the two bricks) and `fourierVar_drop_at_critical_point` (their assembly).

Away from critical points every member keeps its sign—a polynomial cannot change sign without a root, by the intermediate value theorem—so V is locally constant (`fourierVar_eq_of_no_root`, inherited verbatim from the Sturm toolkit). That is why the staircase is flat between criticals. All the action is at a critical point c, and the new difficulty is the vanishing block.

The two bricks. The tower has one feature that replaces coprimality: each member is the derivative of the one above it, $p^{(k+1)} = (p^{(k)})'$. So if $p^{(k)}(c) = 0$ while $p^{(k+1)}$ keeps a constant nonzero sign s just off c, then $p^{(k)}$ is strictly monotone there, and its sign just off c is forced—*the same* as $p^{(k+1)}$ on the right, the *opposite* on the left. No Taylor expansion, no multiplicity bookkeeping: one application of the mean value theorem.

The two monotonicity bricks (recipe)

If $q(c) = 0$ and q' keeps the constant nonzero sign s on $(c, z]$, then $\operatorname{sign} q(z) = s$ (`sign_eval_right_vanish`). If instead q' keeps sign s on $[z, c)$, then $\operatorname{sign} q(z) = -s$ (`sign_eval_left_vanish`). Apply this up a vanishing block, from the first *non*-vanishing member above it downward: **on the right of c each vanishing member copies the sign above it**, so the block collapses to a single sign and contributes *no* sign change; **on the left each vanishing member takes the opposite sign**, so the block *alternates* and contributes one sign change per member.

Counting the consequences gives the local drop. Just right of c the tower's sign list is the list at

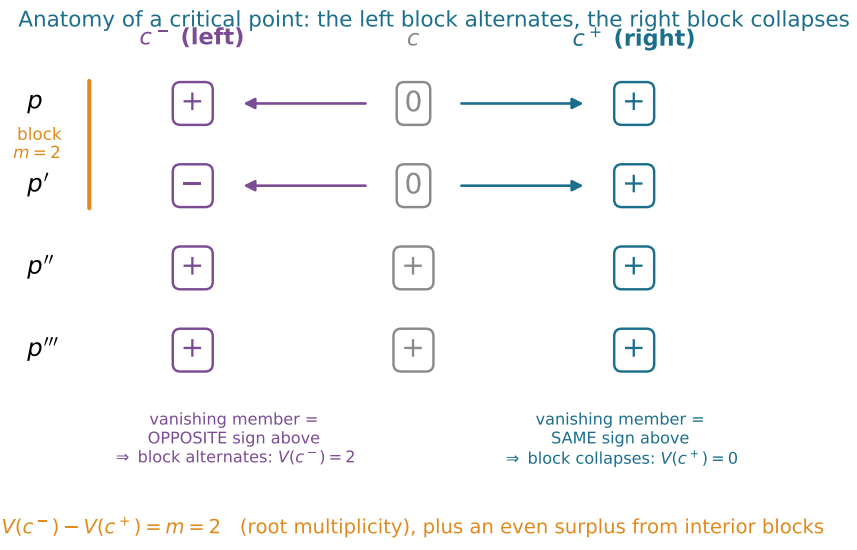

Figure 3: Anatomy of a critical point, for the double root $c = 1$ of p_A (the block p, p' vanishes, $m = 2$). Reading each sign just off c from the member above it: on the right (teal) a vanishing member copies the sign above—the block collapses, $\mathrm{V}(c^+) = 0$; on the left (plum) it takes the opposite sign—the block alternates, $\mathrm{V}(c^-) = 2$. Hence $\mathrm{V}(c^-) - \mathrm{V}(c^+) = m$, the multiplicity, plus an even amount from any interior blocks. Signs are exact from the tower of p_A.

c with each vanishing entry replaced by the sign of its first nonvanishing successor—an operation on bare sign lists I call `Rseq`; just left of c it is the same with *alternating* signs, `Lseq`. The right list has the *same* number of sign changes as the list at c (`Rseq_spec`: $\mathrm{V}(c^+) = \mathrm{V}(c)$, the right side carries no information); the left list has that many *plus* the length of the leading vanishing run *plus* an even surplus from interior blocks (`Lseq_spec`). Writing m for the leading run and packaging the surplus as $2e$,

$$\mathrm{V}(c^-) \;=\; \mathrm{V}(c^+) \;+\; m \;+\; 2e.$$

Figure 3 shows the mechanism on the double root of p_A, where the block p, p' gives $\mathrm{V}(1^-) - \mathrm{V}(1^+) = 2$. The last step is to recognize m: the leading run of vanishing members at c has length exactly the *root multiplicity* of p at c, because in characteristic zero the order of the first nonvanishing derivative *is* the multiplicity. Mathlib knows this (`lt_rootMultiplicity_iff_isRoot_iterate_derivative`), and the roots of p in $(a, b]$ that sit at c number exactly that multiplicity (`count_roots`). So $m = \#\mathrm{roots}(a, b]$ at a genuine root, and $m = 0$ at a critical point that is not a root—in which case the whole drop is the even surplus, exactly Example B. The additive form $\mathrm{V}(a) = \mathrm{V}(b) + \#\mathrm{roots} + 2e$ packages the inequality and the parity into one statement that will add cleanly across the interval.

The bead this hands on is the one the machine cares about most: *this whole picture is a sentence on paper—"the left block alternates and the right collapses"—but where, made formal, does it resist?*

4 Q3: where does the proof resist being made formal?

History bead. On paper the block analysis is a picture; made formal, it is the one genuinely structural step, because the alternation and the collapse must be performed *along a list whose*

shape is recursive, and the hypotheses that justify each step—which member vanishes, what sign its neighbour has—are threaded through the tower's own structure. The device that resolves it, the pair `Rseq`/`Lseq` together with their specifications, is the part of this development with the least direct textbook antecedent: not a new theorem, but the proof engineering that keeps the bookkeeping honest. It is the tower's analogue of the `FlankReduce` relation that did the same job for Sturm [15].

The difficulty is precise. To compare $\mathrm{V}(a)$ with $\mathrm{V}(b)$ across c, one must perform the collapse-on-the-right and alternate-on-the-left on the evaluated sign lists, and the rule for each entry depends on whether *that* member vanishes at c and on the sign of the member above it. The combinatorics of rewriting a list and the algebra of why each rewrite is legal are entangled, and an honest proof separates them.

The separation is two list operations on bare signs, defined by recursion and proved correct without ever mentioning a polynomial.

The decoupling (recipe)

On a list s of signs (entry 0 = a vanishing member, ± 1 = a kept one), `Rseq` replaces each 0 by the value below it and `Lseq` replaces it by the negation of the value below it. Two facts then divide the labour. *Combinatorial:* `Rseq_spec` shows the right list has the same sign-change count as s, and `Lseq_spec` shows the left list has that count plus the leading-zero run plus an even number—this is pure list induction carrying a head sign-parity invariant, no polynomials. *Structural:* `signs_right_eq_Rseq` and `signs_left_eq_Lseq` show that the tower's evaluated sign list at b (resp. a) *is* `Rseq` (resp. `Lseq`) of the list at c—this is the induction over the tower, fed by the two monotonicity bricks and by nothing else.

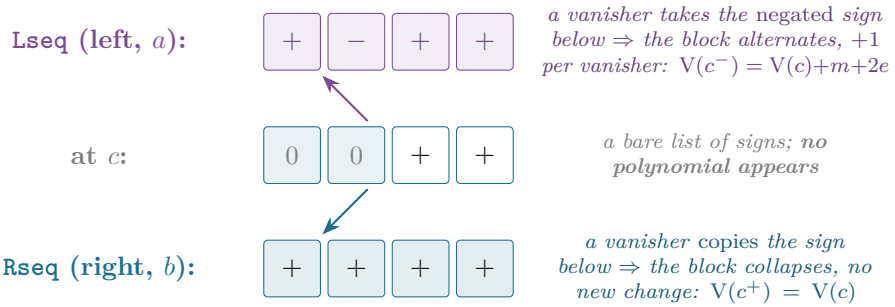

Figure 4: The decoupling of Section 4, at the level of bare sign lists (here the block $0, 0$ of the double root of p_A). `Rseq` and `Lseq` are pure list operations—a vanishing entry copies, resp. negates, the surviving sign below it—and their counting specifications (`Rseq_spec`, `Lseq_spec`) are proved by list induction with no polynomial in sight. The structural lemmas `signs_right_eq_Rseq`/`signs_left_eq_Lseq` are the only bridge back to the tower, and they are fed solely by the two monotonicity bricks.

Three smaller frictions are worth naming, because each is a place the machine refused a hand-wave. *First*, the tower's recursive structure had to be made explicit: that consecutive members are related by differentiation is the hypothesis `List.IsChain`, and the bricks consume it one step at a time as the induction walks down a block. *Second*, the multiplicity step is not a definition but a theorem, and a nontrivial one—that the leading vanishing run equals the root multiplicity rests on Mathlib's characterization of multiplicity by iterated derivatives, and on identifying the first *non*-vanishing member as the top of the tower, a nonzero constant, so the run always terminates.

Third, a small but real Lean trap: rewriting under `List.getLast`, whose value carries a proof that the list is nonempty, is not a plain `rw`; the dependency forces `simp` or an explicit congruence.

What the machine made me state correctly. As with Sturm, formalizing forbade the genericity hand-wave—"slide the endpoints to non-critical points"—and rewarded stating the local drop at the weakest honest level. `fourierVar_drop_at_critical_point` is proved for a critical point c anywhere in the *closed* interval $[a, b]$, endpoints included. The endpoint cases are not special pleading: when $c = a$ the interval $(a, b]$ excludes it and the local count is the even surplus with no multiplicity term; when $c = b$ the same additive identity holds with the multiplicity carried by the leading run. Both fall out of the one statement, discharged by the same lines as the interior case—the principle the companion paper on Newton's inequalities [16] also drew: the weakest honest generality dissolves edge cases rather than multiplying them.

From one step to the whole staircase. The global theorem is then an induction on the number of *distinct* critical points in $(a, b]$ (the roots of the product of all tower members, a nonzero polynomial, hence finite). Peel off the largest critical point c; choose a non-critical separator d just below it; apply the per-point drop on $[d, b]$, where c is the only critical point; recurse on $[a, d]$. The half-open interval splits as $(a, d] \sqcup (d, b]$, the multiplicity-aware root count adds (`rootCount_split`), and the two additive existentials combine into one. The bead handed on: *now that the theorem is certified, what does the library get for free?*

5 Q4: what becomes free once the theorem is certified?

History bead. Budan's rule and Fourier's theorem are, between them, the bridge from Descartes' coefficient count to interval methods; the modern algorithmic descendants are the continued-fraction and bisection real-root isolators of Vincent, Collins and Akritas [7, 6]. The parity refinement is the part that explains the others.

Descartes, and why it overcounts evenly. Descartes' rule is the $b \to \infty$ shadow of Budan–Fourier: the signs of $p^{(k)}$ at large positive x are the signs of the leading coefficients, and the sign variations there are exactly the coefficient sign variations Mathlib already counts. Budan–Fourier on $(0, \infty)$ then says: the number of positive roots is at most the coefficient sign variations, and differs from it by an even number. That even number is the content—it is why Descartes' rule, applied to a polynomial with v sign changes, never reports a parity error: **the unseen roots come in conjugate pairs, and each pair contributes 2 to the gap**. Example B is the smallest instance: $x^2 + 1$ has $\mathrm{V} = 2$ sign variations on the half-line and zero real roots, the gap a single conjugate pair. The Lean count used here and Mathlib's `Polynomial.signVariations` are, after unfolding, the same kind of object, so the toolkit transfers; what the tower adds over the coefficients is precisely the interval and the localization of the even gap. Figure 5 makes the accounting visible: the bound splits into the real roots actually present and twice the conjugate pairs the interval cannot see.

The companion of Sturm. The two theorems now sit side by side in the same Lean development, built on one sign-variation toolkit: Sturm's exact count of *distinct* roots on the remainder chain [15], and Budan–Fourier's multiplicity-aware bound on the derivative tower. The shared core—local constancy, the recursive sign-change count, the leading-pair recursion—was written once and used twice. Their difference is now a formal object too: Sturm's chain is coprime and its local analysis is a single head-drop; the tower is not, and its local analysis is the block collapse of Section 3.

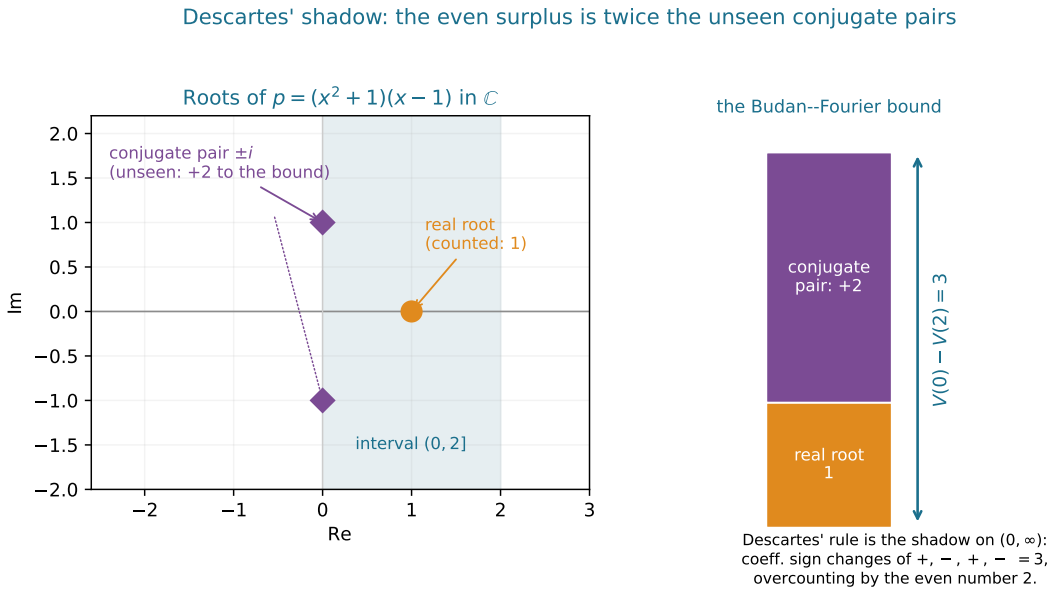

Figure 5: Descartes' shadow. For $p = (x^2+1)(x-1)$ on $(0, 2]$ the tower drop is $\mathrm{V}(0) - \mathrm{V}(2) = 3$: the single real root $x = 1$ contributes 1, and the unseen conjugate pair $\pm i$ contributes the even remainder 2. Read on $(0, \infty)$ this is Descartes' rule—three coefficient sign changes $+, -, +, -$, overcounting the one positive root by an even number. The surplus is always even because the complex roots of a real polynomial come in conjugate pairs. Counts asserted exactly before drawing.

6 What a certified Budan–Fourier enables

Beyond Descartes, the certified theorem is the first verified step of the classical bridge to real-root isolation. A subdivision method that bisects an interval and reads $\mathrm{V}(a) - \mathrm{V}(b)$ on each half has, in this theorem, a machine-checked guarantee that the reported number is an upper bound on the roots and that it drops to the true count as the intervals shrink around simple roots; the multiplicity-aware version is exactly what isolation of multiple roots needs. Wenda Li's Isabelle development [9] took this route to a verified root-counting procedure; the Lean pieces here are the foundation on which the same could be built over Mathlib. Nothing in this paper builds the isolator—that is named honestly among the open questions—but the load-bearing lemma it would stand on is now in the library.

7 The building blocks

Table 2 gathers the load-bearing results, all `sorry`-free in `BudanFourier.lean`; Table 3 reads Example B off its tower, the smallest witness of the even surplus; Table 4 measures the development. The dependency runs as the reader's map (Figure 1): the two monotonicity bricks feed the per-member sign recursions, which feed `signs_right_eq_Rseq`/`signs_left_eq_Lseq`; these, with the pure-list `Rseq_spec`/`Lseq_spec` and the multiplicity identification, give the local drop `fourierVar_drop_at_critical_point`; and the global induction over critical points assembles `budan_fourier`.

Table 2: The load-bearing results, all `sorry`-free in `BudanFourier.lean` (Lean 4 / Mathlib). `#print axioms BudanFourier.budan_fourier` reports only `propext`, `Classical.choice`, `Quot.sound`. To the best of my knowledge this is the first formalization of the Budan–Fourier theorem in Lean.

Lean name	Statement	Status
`budan_fourier`	$\#\text{roots}(a,b] \le \mathrm{V}(a) - \mathrm{V}(b)$ and $\mathrm{V}(a) - \mathrm{V}(b) - \#\text{roots}$ even	proven
`fourierVar_drop_at_critical_point`	$\exists e,\ \mathrm{V}(a) = \mathrm{V}(b) + \#\text{roots}(a,b] + 2e$ (local core)	proven
`fourierVar_eq_signVarAt`	$\mathrm{V}_p(x) =$ `signVarAt` (`fourierSeq` p) x (bridge, `rfl`)	proven
`signs_right_eq_Rseq`	signs at $b =$ `Rseq` of signs at c (right collapses)	proven
`signs_left_eq_Lseq`	signs at $a =$ `Lseq` of signs at c (left alternates)	proven
`Rseq_spec` / `Lseq_spec`	$\mathrm{V}(c^+) = \mathrm{V}(c)$; $\mathrm{V}(c^-) = \mathrm{V}(c^+) + m + 2g$ (combinatorial)	proven
`fourierSeq_isChain`	the tower is a derivative chain $p^{(k+1)} = (p^{(k)})'$	proven
`sign_eval_{right,left}_vanish`	the two monotonicity bricks (sign just off a root)	proven

x	$p_B = x^2+1$	$p_B' = 2x$	$p_B'' = 2$	$\mathrm{V}(x)$
-2	$+$	$-$	$+$	2
-1	$+$	$-$	$+$	2
0 (p_B'=0, not a root)	$+$	0	$+$	0
1	$+$	$+$	$+$	0
2	$+$	$+$	$+$	0

Table 3: Example B, $p_B = x^2 + 1$, with no real root. V drops by 2 at $x = 0$—a zero of the interior member p_B', *not* a root of p_B. Thus $\#\text{roots}(-2, 2] = 0 < \mathrm{V}(-2) - \mathrm{V}(2) = 2$ and the surplus is even: the signature of a conjugate pair, and exactly the `fourierVar_drop_at_critical_point` case with witness $e = 1$.

Table 4: The development measured. Compile time is for checking `BudanFourier.lean` against a prebuilt Mathlib and the imported `Sturm` library, whose sign-variation toolkit is reused.

Quantity	Value
Lines in `BudanFourier.lean`	940
Theorems / definitions	28
`sorry` / `admit` / `native_decide`	0
Axioms beyond Lean's core	0
Reused from `Sturm.lean`	sign-variation toolkit
Compile time (warm Mathlib)	≈ 8 s

8 The boundary of the contribution

I want the limits stated without varnish.

Not first in any system. The Budan–Fourier theorem was formalized in Isabelle/HOL by Wenda Li in 2018–2019 [9], with multiplicity, and used there to drive a verified root-counting procedure; Li also extended a prior Isabelle formalization of Sturm's theorem to count with multiplicity. The present work is, to the best of my knowledge, the first proof in *Lean*, based on searches of Loogle and a `grep` of Mathlib in June 2026. The value is the Lean-and-Mathlib formalization and its reusable machinery, not priority over Isabelle.

The field and characteristic. Everything is over $\mathbb{R}$. The multiplicity step uses characteristic zero in an essential way—it is what makes the order of the first nonvanishing derivative equal the root multiplicity—so the proof as written does not transfer verbatim to positive characteristic, where that identity fails.

What is and is not counted. The theorem counts roots with multiplicity and bounds them by the sign-variation drop, with the even-parity refinement. It does *not* compute the roots, does not produce a certificate of the count for a downstream tool, and does not build a root isolator; those are named among the open questions.

What is reused versus new. The sign-variation toolkit—local constancy, `signChanges`, the leading-pair recursion—is imported from the Sturm development [15] and not re-derived. New here are the two monotonicity bricks, the `Rseq`/`Lseq` decoupling and their specifications, the identification of the leading run with the multiplicity, and the assembly into the local drop and the global theorem.

9 Reflective questions, for the next bead

A string of beads is meant to be continued. I close with the questions I would rather carry forward than pretend to have settled—each a place where this work stops short, and each, I think, the start of another paper.

1. *From bound to certificate.* The theorem says the drop *bounds* the count and shares its parity; the even surplus measures how far the bound overshoots. Can the surplus be made into a *computation*—a verified procedure that, by bisecting until the surplus vanishes, returns the exact multiplicity-aware root count, and a certificate a skeptical tool can re-check? How much of the continued-fraction machinery of Vincent and Akritas [7, 6] would the honest bridge want?

2. *One theorem behind two counts.* Descartes' coefficient count and the tower count are, after unfolding, the same kind of object (Section 5), and Sturm's chain count is built on the same toolkit ([15]). Is there a single Lean statement—a sign-variation principle on an abstract "admissible sequence"—of which Descartes' bound, Budan–Fourier's interval bound, and Sturm's exact count are three instances, so the library holds the local lemma once rather than three times?

3. *What the decoupling really is.* `Rseq`/`Lseq` separated the combinatorics of rewriting a sign list from the algebra licensing each rewrite, exactly as `FlankReduce` did for Sturm. Two different chains, the same shape of argument: is "a sign-change tally transformed by a locally-licensed list rewrite" a reusable lemma in its own right, or did it merely fit twice? I suspect the former and have not proved it.

4. *The block, beyond characteristic zero and beyond one variable.* The multiplicity step leans on characteristic zero; the tower itself does not. What is the right statement over a general field, where the leading run need not equal the multiplicity? And is there a Bernstein-basis or multivariate analogue of the block collapse—the place where root counting on the line meets the geometry of several variables?

Reproducing

```
git clone https://github.com/karlesmarin/godsil-gutman-lean
cd godsil-gutman-lean && lake exe cache get
```

```
lake build Sturm           # the imported sign-variation toolkit
lake env lean BudanFourier.lean
```

Lean `v4.30.0-rc2`, Mathlib pin `701fb6e9`. The theorem is `BudanFourier.budan_fourier` in `BudanFourier.lean`; its supports are in Table 2. It is checked by `#print axioms` to depend only on `propext`, `Classical.choice`, `Quot.sound`. The worked examples of Figures 2 and 3 are re-derived, and every sign and count asserted against exact arithmetic, before the figures are drawn (`figures/budan_fig1_tower.py`, `figures/budan_fig2_critical.py`).

References

[1] R. Descartes, *La Géométrie* (1637); the rule of signs bounds the number of positive roots by the number of sign changes of the coefficients.

[2] F. D. Budan de Boislaurent, *Nouvelle méthode pour la résolution des équations numériques*, Paris (1807; 2nd ed. 1822).

[3] J. Fourier, *Analyse des équations déterminées*, Firmin Didot, Paris (posthumous, 1831).

[4] C. Sturm, Mémoire sur la résolution des équations numériques, *Mémoires présentés par divers savants à l'Académie Royale des Sciences* **6** (1835) 271–318; presented in 1829.

[5] A. G. Akritas, Reflections on a pair of theorems by Budan and Fourier, *Mathematics Magazine* **55** (1982) 292–298.

[6] G. E. Collins, A. G. Akritas, Polynomial real root isolation using Descartes' rule of signs, in *Proc. SYMSAC '76*, ACM, 1976, 272–275.

[7] A. Alesina, M. Galuzzi, A new proof of Vincent's theorem, *L'Enseignement Mathématique* **44** (1998) 219–256.

[8] S. Basu, R. Pollack, M.-F. Roy, *Algorithms in Real Algebraic Geometry*, 2nd ed., Algorithms and Computation in Mathematics **10**, Springer, 2006 (sign-variation theory, Ch. 2).

[9] W. Li, Counting Polynomial Roots in Isabelle/HOL: A Formal Proof of the Budan–Fourier Theorem, in *Proc. 8th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP 2019)*, ACM, 2019, 52–64; preprint `arXiv:1811.11093`; Archive of Formal Proofs entry "The Budan–Fourier Theorem and Counting Real Roots with Multiplicity" (2018), https://isa-afp.org/entries/Budan_Fourier.html.

[10] W. Li, L. C. Paulson, A formal proof of the Sturm–Tarski theorem, *Archive of Formal Proofs* (2014), https://isa-afp.org/entries/Sturm_Tarski.html.

[11] M. Eberl, Sturm's Theorem, *Archive of Formal Proofs* (2014), https://isa-afp.org/entries/Sturm_Sequences.html.

[12] C. Cohen, Construction of real algebraic numbers in Coq, in *Interactive Theorem Proving (ITP 2012)*, LNCS **7406**, Springer, 2012, 67–82.

[13] The mathlib Community, The Lean mathematical library, in *Certified Programs and Proofs (CPP 2020)*, ACM, 2020, 367–381.

[14] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction (CADE 28)*, LNCS **12699**, Springer, 2021, 625–635.

[15] C. Marín, *The Staircase of Signs: Sturm's Root-Counting Theorem, Machine-Checked in Lean 4*, Zenodo, 2026, `doi:10.5281/zenodo.20707348`.

[16] C. Marín, *The Inward Bow of a Real-Rooted Polynomial: Newton's Inequalities, Machine-Checked in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.20693064.

[17] C. Marín, *Random Signs into Matchings: A Godsil–Gutman Identity, Formalized in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.20517350.